

R3.01

CM 1 : Intro et HTTP

Objectifs

L'objectif de cette ressource est de poursuivre l'apprentissage de la programmation autour de technologies web. Cette ressource met en situation de développement à partir de spécification, ce qui est la suite logique de l'apprentissage du développement.

Choix Techniques

- Java Entreprise Edition
- Java Persistant Api
- Spring boot

Cours

- 6 CM

- Intro
- Servlet
- JSP

- JPA
- Spring Boot
- Ouverture

Evaluations

S46 : Servlet + JSP
+ JPA

S50 : MVC

S1 SAE :
Oral + écrit

Semaine	cm	td	tp
35			
36			
37		Rentr	
38	1	ée	
39			
40			
41	1	2	R3.01
42	1	3	web
43	1	3	
44	1	3	
45			
46	1	3	
47		2	
48		3	
49		3	
50		3	
51			

Les références

JEE

<https://www.oracle.com/fr/java/technologies/java-ee-glance.html>

Servlet

<https://www.oracle.com/java/technologies/java-servlet-tec.html>

JSP

<https://www.oracle.com/java/technologies/jspt.html>

JPA

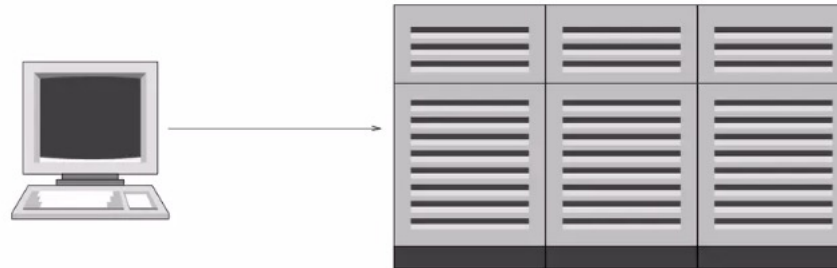
<https://www.oracle.com/java/technologies/persistence-jsp.html>

Spring Boot

<https://spring.io/projects/spring-boot>

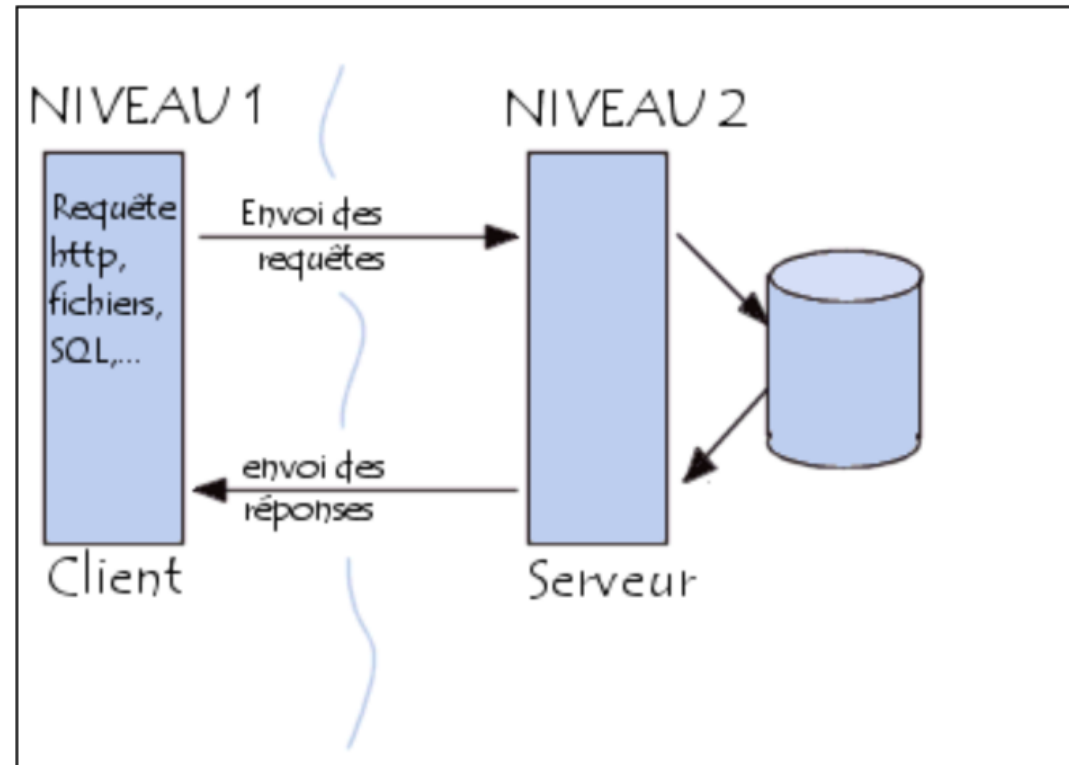
Architecture Applicative – 1 tiers

Année 70, architecture main frame



Architecture Applicative – 2 tiers

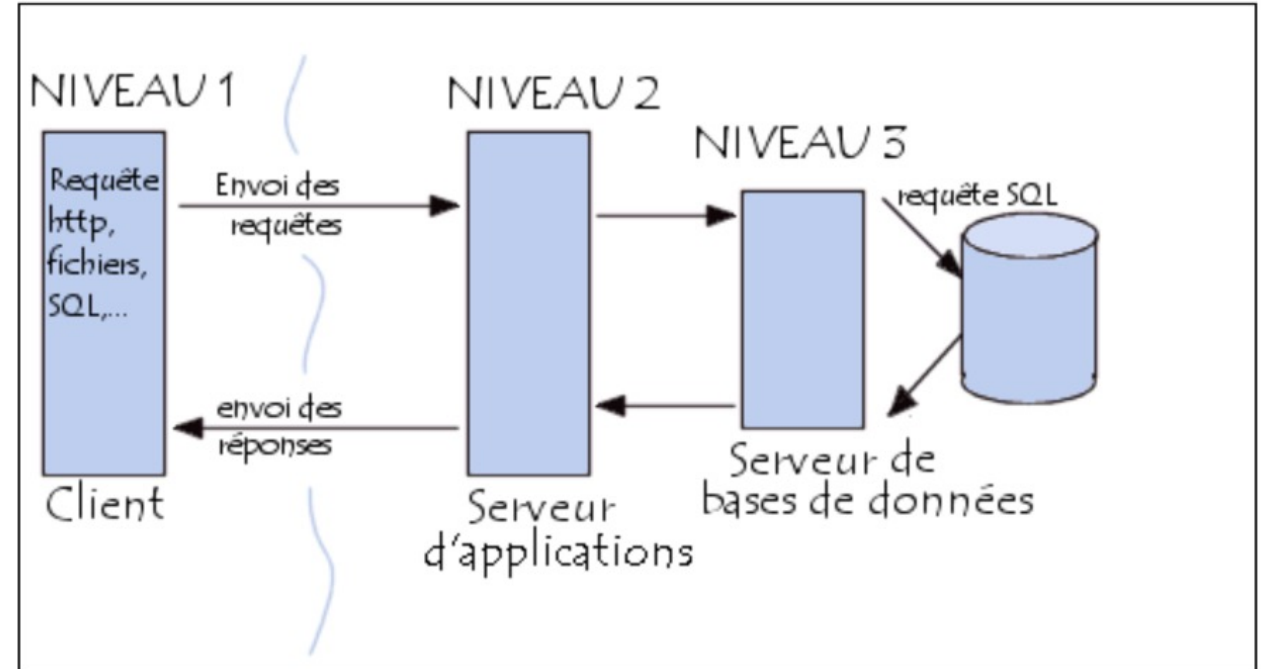
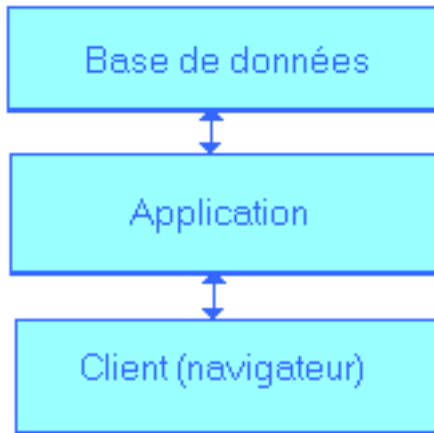
Année 80, architecture client serveur



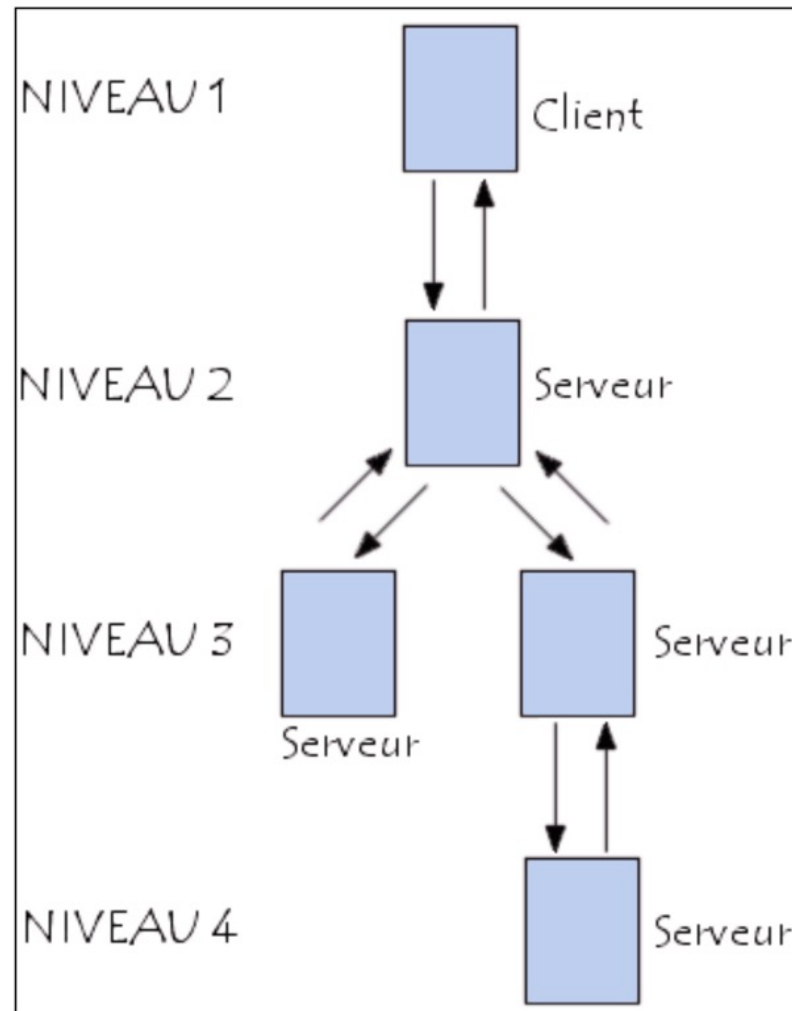
Architecture Applicative – 3 tiers

Architecture en couche

- la couche présentation.
- la couche application
- la couche donnée ou métier.

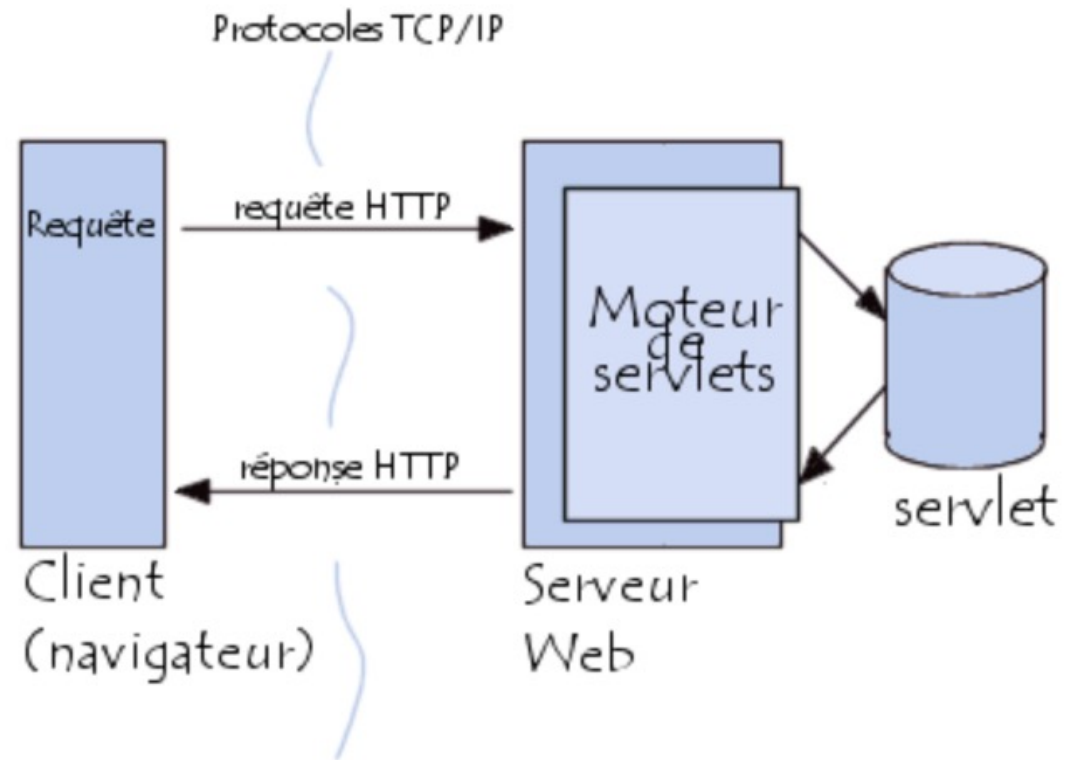


Architecture Applicative – n tiers



Client serveur en java

- Servlet (HttpServlet)
- Moteur de servlet
 - Tomcat
 - Jetty
 - JBoss
 - Glassfish
 - Oracle Application Server



Client serveur 😊

Il faut :

- connaître les bases du protocole HTTP
- connaître les bases de HTML
- savoir programmer objet (Java)
- connaître le SQL
- avoir des bases en configuration de serveur web et de SGBD

Le protocole HTTP

Hypertext Transfer Protocol :

- Protocole de communication client-serveur
- Over TCP port 80 ou 443 (HTTPS)
- Pour la requête comme pour la réponse une entête (header) et un corps (body) optionnel.
- Protocole sans état (stateless protocol)
- Des méthodes GET, POST (PUT, DELETE, HEAD, OPTIONS, TRACE, ...)
- Un grand nombre de serveur : apache, nginx, nodeJS, ...

Le protocole HTTP : GET

Requête une entête sans corps

GET / HTTP/1.1
Host: perdu.com
User-Agent: curl/7.79.1
Accept: */*

Réponse : entête

HTTP/1.1 200 OK
Date: Tue, 06 Sep 2022 19:39:17 GMT
Server: Apache
Upgrade: h2
Connection: Upgrade
Last-Modified: Thu, 02 Jun 2016 06:01:08 GMT
ETag: "cc-5344555136fe9"
Accept-Ranges: bytes
Content-Length: 204
Cache-Control: max-age=600
Expires: Tue, 06 Sep 2022 19:49:17 GMT
Vary: Accept-Encoding, User-Agent
Content-Type: text/html

Réponse : corps

```
<html>
<head>
<title>Vous Etes Perdu ?
</title>
</head>
<body>
<h1>Perdu sur l'Internet ?
</h1>
<h2>Pas de panique, on va vous aider
</h2><strong><pre>  * <----- vous &ecirc;tes
ici</pre></strong>
</body>
</html>
```

Le protocole HTTP : GET avec paramètres

Dans l'entête sous la forme

ressource?param1=val1¶m2=val2

[http://localhost/~jub/php/td1/get_post.php?age=18&nom=le%20coq
&pr%C3%A9nom=correntin](http://localhost/~jub/php/td1/get_post.php?age=18&nom=le%20coq%20et%20pr%C3%A9nom=correntin)

Le protocole HTTP : POST

```
curl -d "param1=value1&param2=value2" -H "Content-Type: application/x-www-form-urlencoded" -X POST "http://localhost/~jub/php/td1/get_post.php"
```

Requête

POST /~jub/php/td1/get_post.php HTTP/1.1

Host: localhost

User-Agent: curl/7.79.1

Accept: */*

Content-Type: application/x-www-form-urlencoded

Content-Length: 27

param1=value1¶m2=value2 *(non affiché sur la version de l'IUT)*

Réponse

HTTP/1.1 200 OK

Date: Tue, 06 Sep 2022 20:53:48 GMT

Server: Apache/2.4.54 (Unix) PHP/8.1.8

X-Powered-By: PHP/8.1.8

Content-Length: 180

Content-Type: text/html; charset=UTF-8

<html> ...

Le protocole HTTP : codes de réponse

Les réponses informatives (100 - 199),

Les réponses de succès (200 - 299),

Les messages de redirection (300 - 399),

Les erreurs du client (400 - 499),

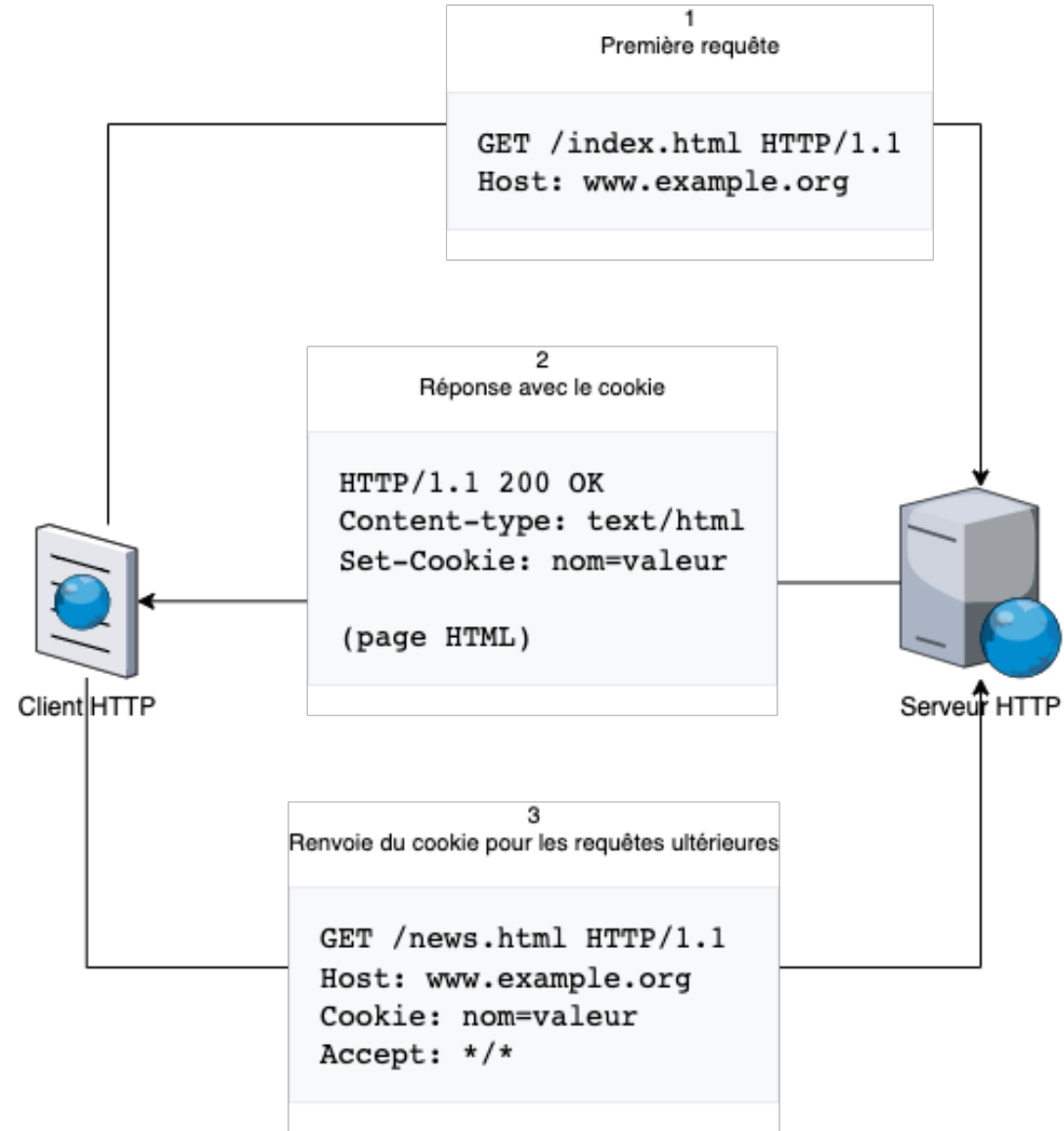
Les erreurs du serveur (500 - 599).

Le protocole HTTP : cookies

Idée maintenir un contexte de connexion pour :

- Gestion des sessions
- Personnalisation
- Le suivi

Le protocole HTTP : cookies



HttpServlet / Cookies

HttpServlet

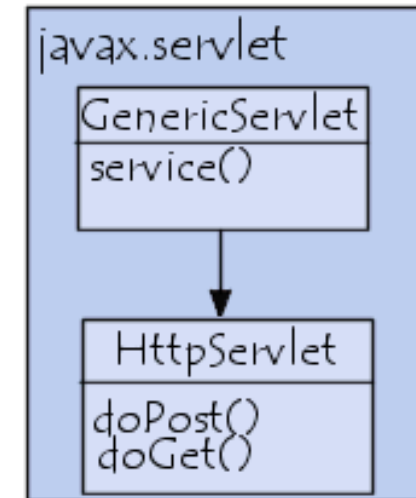
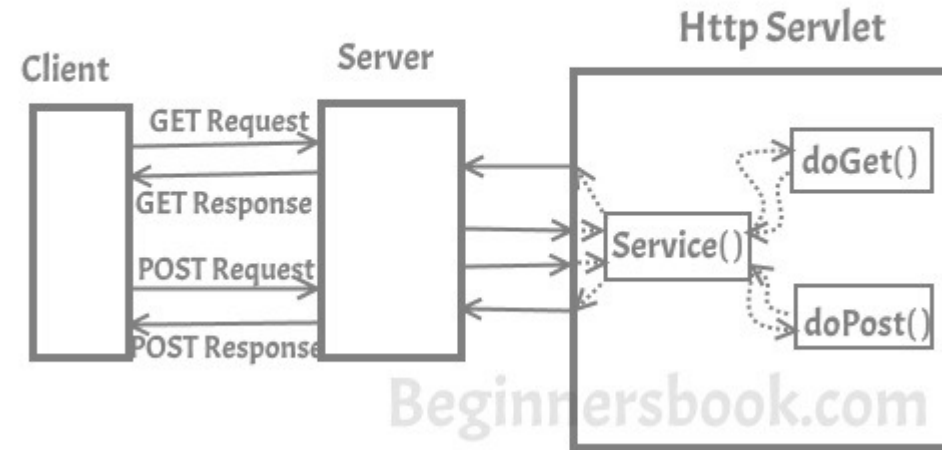
- doGet(), doPost(), ...
- Un objet réponse et un objet requête

Cookie

- Création
 - Instancié
 - Configuré
 - Ajouté à la réponse
- Récupération dans la requête

HttpSession

- Obtenir
- Définir, utiliser un attribut



Hello world

```
package iut.nantes.demo;

import java.io.*;

import jakarta.servlet.http.*;
import jakarta.servlet.annotation.*;

@WebServlet(name = "helloServlet", value = "/hello-servlet")
public class HelloServlet extends HttpServlet {
    private String message;

    public void init() {
        message = "Hello World!!";
    }

    public void doGet(HttpServletRequest request, HttpServletResponse response) throws IOException {
        response.setContentType("text/html");

        // Hello
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h1>" + message + "</h1>");
        out.println("</body></html>");
    }

    public void destroy() {
    }
}
```

Environnement d'exécution à l'IUT (podman)

Podman (docker) (étudié en S4) :

- permet la conteneurisation
 - Un fichier de base docker-compose.yaml un outil de définition et d'exécution d'applications multi-conteneurs
 - Un fichier spécifique pour créer une image docker tomcat/dockerfile
- Des scripts contenant des commandes de base
 - ./create.sh : création du conteneur nommé tomcat_container qui fait tourner un tomcat, le port exposé est 8888 et le répertoire local tomcat/webapps est monté dans le conteneur /usr/local/tomcat/webapps
 - ./down.sh : pour tomber le conteneur
 - ./terminal.sh : pour obtenir un terminal dans le conteneur
 - podman logs tomcat_container pour accéder aux logs du conteneur

Déploiement sur un tomcat

- War (Web ARchive)
 - À générer et à déposer dans le répertoire webapps de tomcat (pas besoin de redémarrer le serveur)
- Utiliser Tomcat Manager (à configurer tomcat-users.xml)
- Passer par un CI/CD

Compiler build.gradle

```
plugins {  
    id 'war'  
}  
  
group 'iut.nantes'  
version '1.0-SNAPSHOT'  
  
repositories {  
    mavenCentral()  
}
```

```
java {  
    sourceCompatibility =  
        JavaVersion.VERSION_21  
    targetCompatibility =  
        JavaVersion.VERSION_21  
}  
  
tasks.withType(JavaCompile) {  
    options.encoding = 'UTF-8'  
}
```

Compiler build.gradle

```
dependencies {  
    compileOnly('jakarta.servlet:jakarta.servlet-api:6.2.0-M2')  
  
}  
  
war {  
    archiveFileName = 'demo.war'  
}
```

Compiler build.gradle

```
tasks.register('deleteMyWar',Delete) {  
  
    doLast {  
        println("Suppression du war :  
./tomcat/webapps/${war.archiveFileName.get()}")  
        println("Done")  
    }  
    delete files("./tomcat/webapps/${war.archiveFileName.get()}")  
  
}
```

Compiler build.gradle

```
tasks.register('copyMyWar',Copy){

    doLast {
        println("Copy ${war.archiveFileName.get()} from ${project.buildDir}/libs to
./tomcat/webapps")
        println("Done")
    }
    from "${project.buildDir}/libs"
    into "./tomcat/webapps"
    include "${war.archiveFileName.get()}"
}
```

À l'IUT gradle.properties

systemProp.http.proxyHost=srv-proxy-etu-2.iut-nantes.univ-nantes.prive

systemProp.http.proxyPort=3128

systemProp.https.proxyHost=srv-proxy-etu-2.iut-nantes.univ-nantes.prive

systemProp.https.proxyPort=3128

gradle

./gradlew

./gradlew tasks

./gradlew war

./gradlew deleteMyWar

./gradlew copyMyWar

Prochain CM

Les servlet en pratiques